

Programação Concorrente e Distribuída

Pool de Threads

Glêdson Elias
gledson@ci.ufpb.br

1



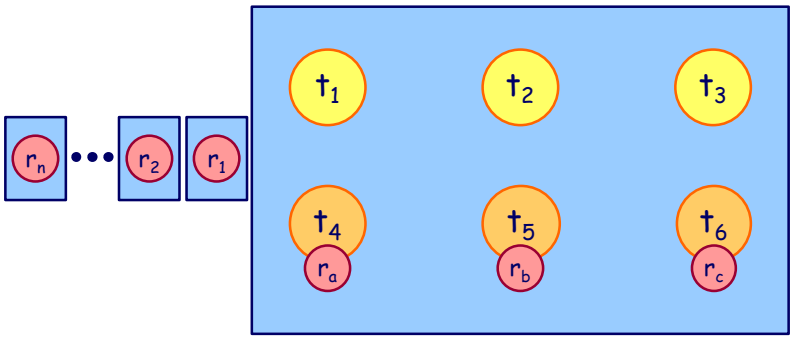
Pool de Threads

- Fundamentos
  - Criação de threads consome recursos de processamento e memória
    - Adequado apenas para aplicações pequenas
    - Deve-se evitar a criação de threads por tarefa
  - Aplicações de larga escala devem adotar Pool de Threads para otimizar a gerência de threads
  - Objeto `Executor` encapsula a criação e a gerência de threads
    - Implementa a interface `Executor`
    - Pool de Threads é um tipo de objeto Executor

2

Pool de Threads

- Fundamentos
  - Visão Conceitual



Pool de Threads

- Interfaces Executor
  - Pacote `java.util.concurrent` define três tipos de interface `Executor`
    - `Executor`
      - Suporta a execução de novas tarefas
    - `ExecutorService`
      - Adiciona facilidades para gerenciar o término de tarefas e do próprio objeto Executor
      - Pode retornar objeto após execução de tarefa
    - `ScheduledExecutorService`
      - Permite a execução de tarefas agendadas e tarefas periódicas

Pool de Threads

- Interface `Executor` — `void execute(Runnable command)`
  - Provê método para executar uma tarefa
  - Dependendo da implementação, pode ...
    - Criar novo thread para executar a tarefa
    - Usar thread existente para executar a tarefa
      - Thread existente reside em Pool de Thread
      - Tarefa pode aguardar em uma fila de espera, caso o thread existente não esteja livre
  - Java provê implementações no pacote `java.util.concurrent`

```
Runnable r = new RunnableTask();
Thread t = new Thread(r);
t.start();
```

```
Runnable r = new RunnableTask();
Executor e = ...
e.execute(r);
```


Pool de Threads

- Interface `ExecutorService`
  - Complementa interface `Executor` com métodos mais versáteis
  - Permite executar objetos `Runnable` e `Callable`
  - Retorna valor após a execução da tarefa
    - Retorno é efetivado por um objeto `Future`
  - Também permite submeter a execução coleções de objetos `Callable`
  - Provê diversos métodos para gerenciar o término (shutdown) do objeto `Executor`

Pool de Threads

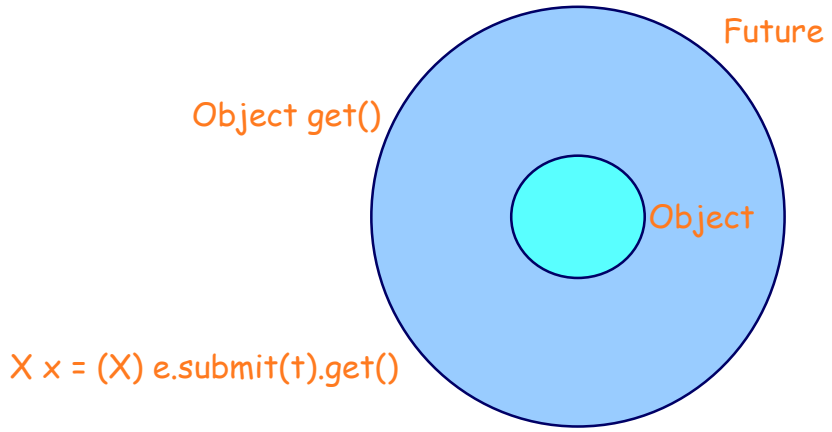
- Interface `ExecutorService`
  - `submit()`
    - Submete uma tarefa a execução
    - Retorna um objeto `Future` que representa o resultado da tarefa
      - Método `get` de `Future` retorna `null` após execução com sucesso de `Runnable`
      - Método `get` de `Future` retorna o resultado após execução com sucesso de `Callable`

```
Future submit(Runnable task)
Future submit(Callable task)
```

```
ExecutorService e = ...
result = e.submit(callable).get();
```


Pool de Threads

- Interface `ExecutorService`



Pool de Threads

- Interface ScheduledExecutorService
  - Complementa interface ExecutorService para permitir temporizar a execução de tarefas

```
ScheduledFuture schedule(Runnable command,
                         long delay, TimeUnit unit)
Executa um objeto Runnable após expirar o retardo especificado
Retorna um objeto ScheduledFuture como resultado da tarefa
Método get do objeto ScheduledFuture retorna null após execução
com sucesso da tarefa

ScheduledFuture schedule(Callable callable,
                         long delay, TimeUnit unit)
Executa um objeto Callable após expirar o retardo especificado
Retorna um objeto ScheduledFuture que como resultado da tarefa
Método get do objeto ScheduledFuture retorna o resultado após
execução com sucesso da tarefa
```


Pool de Threads

- Interface ScheduledExecutorService
  - Complementa interface ExecutorService para permitir repetir a execução de tarefas

```
ScheduledFuture scheduleAtFixedRate(Runnable command,
                                    long initialDelay,
                                    long period, TimeUnit unit)
Tarefa mais longa que o período retarda a execução subseqüente

ScheduledFuture scheduleWithFixedDelay(Runnable command,
                                       long initialDelay,
                                       long delay, TimeUnit unit)
Executa periodicamente um objeto Runnable após expirar o retardo
inicial especificado
Retorna um objeto ScheduledFuture que representa o resultado
pendente da tarefa
```

```
initialDelay; initialDelay + period; initialDelay + 2*period
```

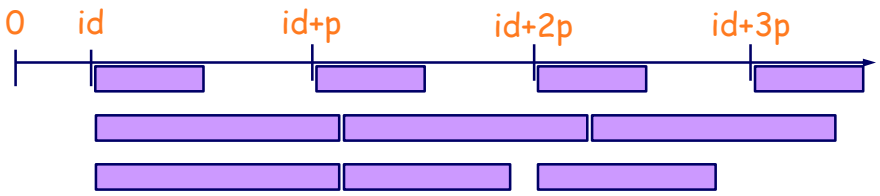
```
initialDelay; endOfExecution + delay; endOfExecution + delay
```

Pool de Threads

- Interface ScheduledExecutorService

```
ScheduledFuture scheduleAtFixedRate(Runnable command,
                                    long initialDelay,
                                    long period, TimeUnit unit)
Tarefa mais longa que o período retarda a execução subseqüente
```

```
initialDelay; initialDelay + period; initialDelay + 2*period
```

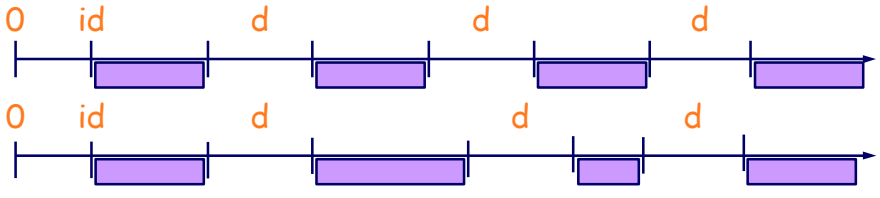


Pool de Threads

- Interface ScheduledExecutorService

```
ScheduledFuture scheduleWithFixedDelay(Runnable command,
                                       long initialDelay,
                                       long delay, TimeUnit unit)
Executa periodicamente um objeto Runnable após expirar o retardo
inicial especificado
Retorna um objeto ScheduledFuture que representa o resultado
pendente da tarefa
```

```
initialDelay; endOfExecution + delay; endOfExecution + delay
```



Pool de Threads

- Suporte em Java
  - Consiste de um conjunto de threads (workers)
    - Threads existem separadamente das tarefas `Runnable` e `Callable`
  - Cada thread pode ser reusado para executar diferentes tarefas
    - Reusar threads minimiza o esforço de criar/liberar threads
  - Implementações da interface `Executor` do pacote `java.util.concurrent` adotam pool de threads

Pool de Threads

- Pool de Threads Fixo
  - Possui um dado número fixo de threads
  - Se um thread é terminado por algum motivo, outro thread é automaticamente criado
  - Tarefas são inseridas em uma fila de espera, caso não existam threads disponíveis
  - Método estático `newFixedThreadPool` da classe `Executors` cria pool de threads fixo

```
ExecutorService e = Executors.newFixedThreadPool(numThreads);
```

Pool de Threads

- Pool de Threads Fixo

```
public class RunnableTask implements Runnable {
   int r;
   public RunnableTask(int r) {
      this.r = r;
   }
   public void run() {
      ...
   }
}
```

```
public class ExecutionTask {
  public static void main(String[] args) {
    int nt = Integer.parseInt(args[0]);
    Executor exec = Executors.newFixedThreadPool(nt);
    int r = 0;
    while (true) {
       exec.execute(new RunnableTask(++r));
    }
  }
}
```


Pool de Threads

- Pool de Threads com Cache
  - Possui um número variável de threads
    - Threads são criados sob demanda e reusados quando disponíveis
    - Threads inativos são removidos (60 segundos)
  - Não adota fila de espera por threads
  - Método estático `newCachedThreadPool` da classe `Executors` cria pool de threads com cache

```
ExecutorService e = Executors.newCachedThreadPool();
```

Pool de Threads

- Pool de Threads com Cache

```
public class RunnableTask implements Runnable {
   int r;
   public RunnableTask(int r) {
      this.r = r;
   }
   public void run() {
      ...
   }
}
```

```
public class ExecutionTask {
  public static void main(String[] args) {
    Executor exec = Executors.newCachedThreadPool();
    int r = 0;
    while (true) {
       exec.execute(new RunnableTask(++r));
    }
  }
}
```


Pool de Threads

- Pool de Threads com Agendamento
  - Possui um dado número fixo de threads
  - Se um thread é terminado por algum motivo, outro thread é automaticamente criado
  - Tarefas são inseridas em uma fila de espera, caso não existam threads disponíveis
  - Método estático `newScheduledThreadPool` da classe `Executors` cria pool de threads com agendamento

```
ScheduledExecutorService e =
              Executors.newScheduledThreadPool(numThreads);
```

Pool de Threads

- Pool de Threads com Agendamento

```
public class RunnableTask implements Runnable {
   int r;
   public RunnableTask(int r) {
      this.r = r;
   }
   public void run() {
      ...
   }
}
```

```
public class ExecutionTask {
  public static void main(String[] args) {
    int nt = Integer.parseInt(args[0]);
    ScheduledExecutorService exec =
                  Executors.newScheduledThreadPool(nt);
    Runnable r = new RunnableTask(0);
    exec.scheduledAtFixedRate(r, 5, 10, SECONDS);
  }
}
```


Exercício IV

- Considerando duas matrizes A e B:
  - Implemente um programa concorrente baseado em pool de threads que calcula AxB, onde cada elemento de AxB é calculado por uma tarefa
    - Mede e exibe o tempo Tp para calcular AxB
    - Exibe a matriz AxB

Qual a relação entre Ts, Tc e Tp?

O desempenho esperado foi observado?

Explique porque o desempenho é ou não é o esperado.

Se não é o esperado, reprojete o uso dos threads para obtê-lo.